

Depuración y Corrección del Pipeline de Diagnóstico Asistido de Biopsias de Mama: Fuga de Información, Desbalance de Clases y Refactorización

1. Diagnóstico

El pipeline usa el conjunto Wisconsin Breast Cancer (Street et al., 1993) de scikit-learn (Pedregosa et al., 2011), enriquecido con dos columnas clínicas sintéticas. Al ejecutarlo se identificaron dos defectos: un `ValueError` por valores infinitos y una fuga de información. Para cada uno se formularon dos hipótesis, se descartó una con evidencia y se confirmó la otra.

Defecto 1: `ValueError` (“Input X contains infinity”)

H1: *cargar_datos()* introduce el valor no finito. H2: *construir_caracteristicas()* lo introduce al calcular *mean texture / num_biopsias_previas*, indefinida cuando ese denominador vale 0.

Validando cada etapa con *validar_datos()*: los datos recién cargados no tenían nulos ni infinitos (H1 descartada). Tras *construir_caracteristicas()* aparecieron 135 valores infinitos (23.7% de las 569 filas), exactamente en las filas con *num_biopsias_previas* == 0 (H2 confirmada). El traceback lo respalda: el error se dispara en la validación de entrada de scikit-learn, antes de que `LogisticRegression` calcule nada — el infinito ya venía corrupto en los datos.

Defecto 2: fuga de información

H1: la relación entre *sesiones_tratamiento_programadas* y *diagnostico* es una correlación fuerte pero incidental. H2: es determinista por diseño, porque las sesiones se programan después de confirmar malignidad — no existen en el momento real de predecir. La verificación descartó H1: 100% de los 212 casos malignos tenían sesiones (>0) y 100% de los 357 benignos tenían 0; no es correlación estadística, es una regla exacta 1 a 1 (H2 confirmada).

2. Corrección

La Tabla 1 compara el accuracy sobre la misma partición (random_state=42) en cada etapa de la corrección. El pipeline original ni siquiera completa el entrenamiento.

Tabla 1

Accuracy antes y después de cada corrección, misma partición

Etapas del pipeline	Accuracy entrenamiento	Accuracy prueba
Original (divide por cero)	ValueError — no entrena	N/A
Fix 1 aplicado (fuga aún presente)	1.0000	1.0000
Fix 2 aplicado (fuga retirada, sin estratificar)	0.7183	0.7413
Versión final (fuga retirada + estratificado)	0.7324	0.7343

El accuracy de 1.0000 tras el fix 1 no es un éxito: en un problema médico real, un accuracy perfecto es señal de fuga, no de calidad. El optimismo que la fuga introducía en el accuracy de prueba fue de +0.2657 (26.6 puntos porcentuales).

3. Desbalance de Clases

Diagnosticos está desbalanceado: 62.7% benigno, 37.3% maligno. comparó entrenar modelo (usar class_weight=False) contra usar class_weight=True, ya sin la fuga, sobre la misma partición estratificada (143 casos de prueba: 53 malignos, 90 benignos).

Tabla 2

Accuracy y recall por clase, con y sin class_weight='balanced'

Configuración	Accuracy	Recall maligno	Recall benigno
class_weight=None	0.7343	0.5094	0.8667
class_weight='balanced'	0.7343	0.6792	0.7667

El accuracy idéntico (0.7343), el recall por clase: con class_weight=None el modelo detecta solo 50.9% de los tumores malignos reales (26 de 53 falsos negativos); con 'balanced' el recall sube a 67.9% (17 falsos negativos), a costa de que la precisión en maligno baje de 0.69 a 0.63.

El accuracy agregado no revela esta diferencia, trata un falso negativo en maligno y falso positivo en benigno como errores equivalentes, la clase mayoritaria (benigno) domina el conteo. Clínicamente no lo son: un falso positivo cuesta una revisión de más (bajo costo); un falso negativo es un tumor maligno sin priorizar (alto costo). Por eso `class_weight='balanced'` vale la pena aunque el accuracy no cambie: mueve la frontera de decisión hacia el error más costoso, una mejora invisible si solo se mira el accuracy agregado.

4. Refactorización

`validar_datos()` estaba definida pero nunca se invocaba desde el pipeline real: `entrenar_modelo()` no la llamaba. Además tenía su propio defecto: referenciaba una variable global `detener` que no existía en ningún lado, así que invocada con datos inválidos habría fallado con `NameError` antes de terminar de validar nada.

Se decidió conectarla en vez de eliminarla: es exactamente el chequeo que habría atrapado el defecto 1 antes de que el infinito llegara a `LogisticRegression.fit()`. Eliminarla habría dejado el pipeline sin ninguna barrera de calidad de datos; el problema nunca fue el diseño de la función, sino que estaba desconectada.

Cambios aplicados: (a) `detener` pasa a ser un parámetro (`detener=True` por defecto), lo que corrige el `NameError`; (b) `entrenar_modelo()` llama a `validar_datos(df, columnas, detener=True)` antes de separar X/y (parámetro `validar=True` por defecto), así que cualquier llamada futura queda protegida sin depender de que alguien invoque la validación a mano. Las pruebas `test_validar_datos_detiene_ante_datos_invalidos` y `test_validar_datos_no_detiene_si_detener_es_false` verifican este cambio.